

NLP Foundations

Clean Text → Useful Features → Meaning

Motivation

Can machines understand text?

Example:

- ▶ I love this movie
- ▶ This product is terrible

Key problem:

Machines need numbers, not words.

Raw Text is Messy

Example:

Thiss movie!!! is sooo GOOD

Problems:

- ▶ Noise
- ▶ Punctuation
- ▶ Inconsistent casing
- ▶ Emojis

Step 1: Cleaning

Goal: Normalize text

Before:

This movie!!! is AMAZING

After:

this movie is amazing

Typical steps:

- ▶ Lowercasing
- ▶ Removing punctuation
- ▶ Removing special characters

Step 2: Tokenization

Split text into words

Input:

this movie is amazing

Tokens:

[this, movie, is, amazing]

Tokens are the basic units of text.

From Text to Numbers

Now we have tokens.

Question:

How do we convert text into numbers?

This leads to **feature extraction**.

Step 3: Bag of Words

Idea: Count words

Vocabulary:

[movie, good, bad]

Examples:

- ▶ movie good $\rightarrow [1, 1, 0]$
- ▶ movie bad $\rightarrow [1, 0, 1]$

Each document becomes a vector.

Limitation of Bag of Words

Example:

- ▶ movie good
- ▶ movie bad

All words are treated equally.

Problem:

Common words dominate.

Step 4: TF-IDF

Idea:

Important words:

- ▶ Frequent in document
- ▶ Rare across documents

Formula:

$$TFIDF = TF \times IDF$$

TF-IDF Intuition

Common words get low weight.

Rare, meaningful words get high weight.

Example:

- ▶ movie → low importance
- ▶ excellent → high importance

TF-IDF Example Setup

Documents:

- ▶ D1: movie is good
- ▶ D2: movie is bad
- ▶ D3: movie is good good

Vocabulary:

[movie, is, good, bad]

Step 1: Term Frequency (TF)

D1: movie is good

$[1, 1, 1, 0]$

D2: movie is bad

$[1, 1, 0, 1]$

D3: movie is good good

$[1, 1, 2, 0]$

Step 2: Document Frequency (DF)

Word	DF
movie	3
is	3
good	2
bad	1

Total documents: $N = 3$

Step 3: Inverse Document Frequency (IDF)

$$IDF(w) = \log \left(\frac{N}{DF(w)} \right)$$

- ▶ movie: $\log(3/3) = 0$
- ▶ is: $\log(3/3) = 0$
- ▶ good: $\log(3/2) \approx 0.405$
- ▶ bad: $\log(3/1) \approx 1.099$

Step 4: TF-IDF Calculation

D1: movie is good

$[0, 0, 0.405, 0]$

D2: movie is bad

$[0, 0, 0, 1.099]$

D3: movie is good good

$[0, 0, 0.81, 0]$

Final TF-IDF Matrix

$$\begin{bmatrix} 0 & 0 & 0.405 & 0 \\ 0 & 0 & 0 & 1.099 \\ 0 & 0 & 0.81 & 0 \end{bmatrix}$$

Rows: Documents

Columns: [movie, is, good, bad]

Key Observations

- ▶ Common words (movie, is) get weight 0
- ▶ Rare words get higher importance
- ▶ Repeated words increase weight

Conclusion:

TF-IDF highlights important words and suppresses common ones.

Limitation of TF-IDF

Example:

- ▶ movie
- ▶ film

TF-IDF treats them as completely different.

Problem:

No understanding of meaning.

Key Question

How do we capture meaning?

We need representations where:

- ▶ Similar words are close
- ▶ Meaning is preserved

Step 5: Word Embeddings

Idea:

Word $\rightarrow$ **Vector**

Each word is represented in a continuous space.

Word2Vec

Words with similar meanings have similar vectors.

Example:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

Now we capture semantic relationships.

From Words to Meaning

Word2Vec captures:

- ▶ Semantic similarity
- ▶ Relationships between words

Example:

- ▶ movie $\approx$ film
- ▶ good $\approx$ excellent

Complete Pipeline

Raw Text → Cleaning → Tokenization → BoW / TF-IDF → Word2Vec

From noise to meaning.

Key Takeaways

- ▶ Cleaning removes noise
- ▶ Tokenization creates structure
- ▶ BoW represents text as counts
- ▶ TF-IDF adds importance
- ▶ Word2Vec captures meaning

Final Thought

Classical NLP:

Counts and statistics

Modern NLP:

Meaning and context